

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e
Ingenierías



Diseño e Implementación de un Procesador de Ciclo Único (Data-Path Tipo R)

Proyecto: DPTR-Fase 1

Materia: Arquitectura de Computadoras

Por:

[ANDREA VALERIA TORRES FIGUEROA](#)
[ESTEFANIA NAVARRO MENDOZA](#)
[ANGEL GAEL GARCIA RAMOS](#)

Docente:

JORGE ERNESTO LOPEZ ARCE DELGADO

I. Introducción y Fundamentos Teóricos

1.1 El "Single Datapath" y la Arquitectura MIPS

La construcción de este procesador se basa en el modelo "Single Datapath", creando una versión simplificada del procesador MIPS diseñada específicamente para decodificar y ejecutar instrucciones de **Tipo R**. El sistema integra elementos básicos previamente analizados como la Memoria, la ALU y el Banco de Registros para gestionar el flujo de datos en un solo ciclo de reloj.

1.2 Análisis de la Instrucción Tipo R

La instrucción posee un ancho de palabra de **32 bits**, segmentada de manera lógica en seis campos fundamentales:

- **OpCode (6 bits):** Código de operación (000000 para Tipo R).
- **RS (5 bits):** Registro fuente 1.
- **RT (5 bits):** Registro fuente 2.
- **RD (5 bits):** Registro destino.
- **Shamt (5 bits):** Cantidad de desplazamiento (Shift Amount).
- **Funct (6 bits):** Especifica la operación aritmética o lógica exacta a realizar.

1.3 La Instrucción SLT y la Operación Ternaria

La instrucción **SLT (Set on Less Than)** es crítica para la implementación de saltos condicionales en software. Su funcionamiento lógico en hardware se optimiza mediante la **operación ternaria**:

$$\$Resultado = (A < B) \ ? \ 32'd1 : 32'd0$$

Esto permite que, si el primer operando es menor que el segundo (usando comparación signada), el registro destino se cargue con un 1, facilitando la lógica de control de flujo.

II. El Proceso de Compilación y Abstracción

El desarrollo de este procesador permite comprender el ciclo de vida de una instrucción:

1. **Código de Alto Nivel:** El programador escribe lógica abstracta.
2. **Compilador:** Traduce la lógica a mnemónicos de ensamblador (ADD, SUB, etc.).
3. **Ensamblado:** Convierte los mnemónicos en código máquina (Hexadecimal/Binario) basado en el formato de instrucción.
4. **Hardware (DPTR):** El procesador recibe los 32 bits y los "separa" físicamente a través de cables para activar los módulos correspondientes.

III. Descripción Arquitectónica de Módulos

El sistema se compone de los siguientes módulos interconectados, diseñados para trabajar en armonía con el banco de registros predefinido:

Módulo	Función Principal	Características Técnicas
Unidad de Control (UC)	Decodificador principal del OpCode.	Genera señales MemToReg, RegWrite (W) y ALUOp.
ALU Control	Escucha a la UC y al campo Funct.	Define la operación exacta enviada a la ALU (3 bits).
Banco de Registros (BR)	Almacenamiento de alta velocidad.	Soporta lectura dual (DR1, DR2) y escritura síncrona habilitada por W.
ALU	Unidad de procesamiento matemático.	Realiza ADD, SUB, AND, OR y SLT con bandera de Zero (ZF).
Memoria (Mem)	Almacenamiento de datos de 64 palabras.	Posee control de lectura/escritura (MemWrite) y direccionamiento de 32 bits.
Mux 2:1	Selector de ruta de datos.	Selecciona si el dato para el registro proviene de la ALU o de la Memoria.

IV. Validación y Pruebas (Testbench)

Para validar el DPTR, se implementó un Testbench con 10 instrucciones (dos de cada tipo solicitado).

4.1 Tabla de Código Máquina (Entradas del TB)

Siguiendo la inicialización de registros (\$R0=10, R1=20, R2=30, R3=40\$):

#	Operación Ensamblador	Código Máquina (Hex)	Resultado Esperado
1	ADD R2, R0, R1	0x00011020	$\$R2 = 10 + 20 = 30\$$
3	SUB R2, R2, R0	0x00401022	$\$R2 = 30 - 10 = 20\$$
5	AND R2, R0, R1	0x00011024	$\$R2 = 10 \text{ \texttt{ AND } } 20 = 0\$$
7	OR R2, R0, R1	0x00011025	$\$R2 = 10 \text{ \texttt{ OR } } 20 = 30\$$
9	SLT R2, R0, R1	0x0001102A	$\$R2 = (10 < 20) \text{ \texttt{ to } } 1\$$

4.2 Análisis de Resultados

Durante la simulación en ModelSim, se otorgó un intervalo de 10ns entre cada instrucción. Este tiempo es vital para asegurar que la lógica combinacional de la ALU y el Banco de Registros se estabilice antes de la siguiente operación.

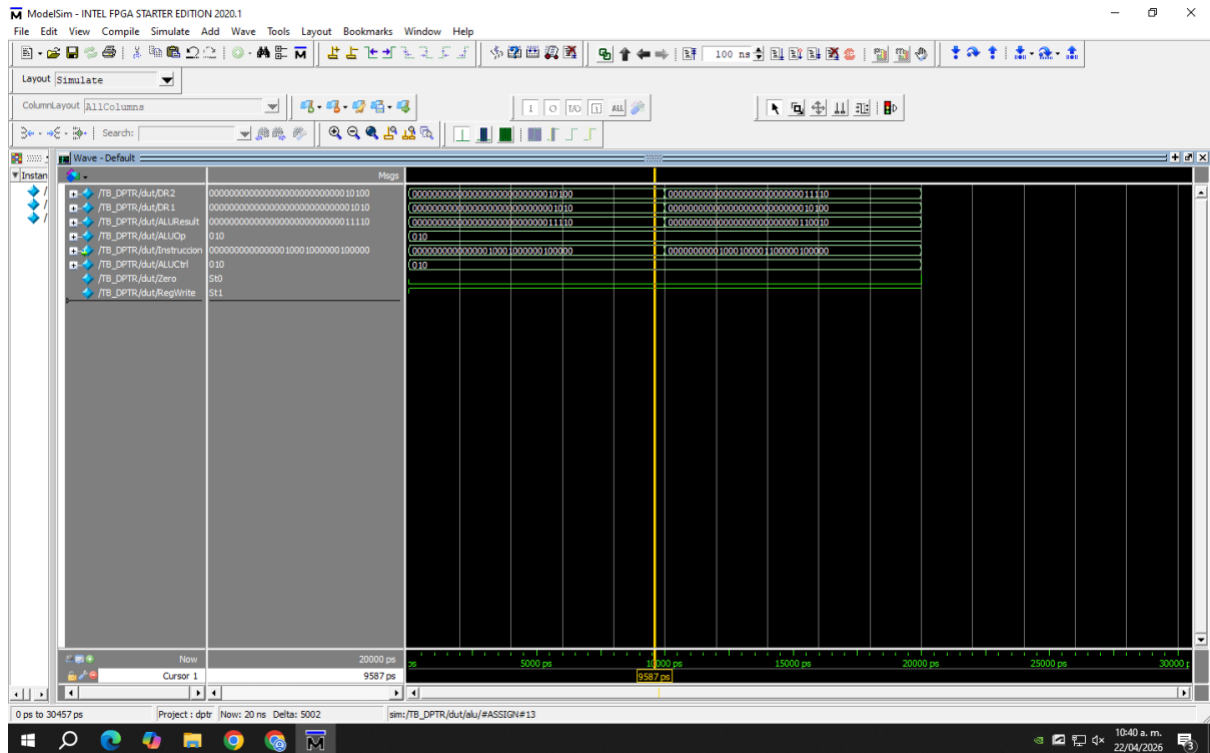


Figura 1. Simulación en ModelSim del Data-Path Tipo R ejecutando las instrucciones del Testbench.

- **Correctitud:** Todas las operaciones aritméticas coinciden con los valores esperados en el simulador en formato decimal.
- **Propagación:** Se otorgaron 10ns entre cada instrucción para asegurar que las señales de control y el banco de registros se estabilizaran correctamente.

V. Nuestro algoritmo

De acuerdo con las notas y requerimientos del proyecto, se determinó omitir la implementación de las instrucciones complejas DIV y MUL en hardware. Para evaluar a fondo la Unidad Aritmético-Lógica (ALU) y el Banco de Registros utilizando exclusivamente operaciones simples, se seleccionó el algoritmo de Raíz Cuadrada Entera por Resta Sucesiva de Números Impares.

5.1 Fundamento matemático

Este algoritmo se fundamenta en un teorema aritmético clásico que establece que la suma de los primeros n números impares consecutivos es exactamente igual a n^2 :

$$1 = 1^2$$

$$1 + 3 = 4 = 2^2$$

$$1 + 3 + 5 = 9 = 3^2$$

$$1 + 3 + 5 + 7 = 16 = 4^2$$

Aplicando el proceso inverso, si a un número entero X se le restan números impares sucesivos (1, 3, 5, 7...) hasta que el residuo sea menor o igual a cero, la cantidad de restas realizadas corresponderá exactamente a su raíz cuadrada entera. Esto permite calcular raíces sin utilizar multiplicadores, divisores ni cálculos cuadráticos en hardware.

5.2 Pseudocódigo de la Lógica de Control

La estructura lógica del algoritmo se plantea mediante un ciclo iterativo controlado por comparaciones simples:

Inicio

```
Numero = Valor_Original // Registro con el número a procesar
Impar = 1                // Primer número impar
Raiz = 0                  // Contador del resultado
```

Mientras (Numero $\geq$ Impar) hacer:

```
Numero = Numero - Impar // Resta el impar actual
Raiz = Raiz + 1          // Incrementa el resultado
Impar = Impar + 2        // Calcula el siguiente impar
```

Fin Mientras

Fin

La elección de este algoritmo se adapta perfectamente a las restricciones del Datapath Tipo R de la Fase 1. Dado que en esta etapa no se cuenta con instrucciones de salto condicional para implementar el ciclo Mientras en hardware, el programa se evalúa “desenrollando” el bucle secuencialmente mediante la combinación de operaciones básicas.

El procesador emula este comportamiento iterativo empleando únicamente la instrucción SUB (para decrementar el valor original) y la instrucción ADD (para incrementar el contador de la raíz y para generar el siguiente número impar).

VI. Validación del Datapath y Resolución en el Testbench

La validación del diseño implicó simular las instrucciones en ModelSim y depurar la transferencia de datos en tiempo real.

6.1 Resolución de Señales Desconocidas (Estado X)

Durante las primeras pruebas del simulador, el bus de datos y las señales de control mostraron un estado rojo (X o desconocido). Se determinó que el módulo de Memoria de Instrucciones (InstructionMemory.v) fallaba al cargar el archivo de texto debido a restricciones de ruteo. La solución consistió en utilizar la ruta absoluta del archivo en la computadora dentro de la función \$readmemb y recompilar los módulos, garantizando así la inyección correcta del código máquina de 32 bits en el Program Counter.

6.2 Corrección de Enrutamiento en ALU Control

Asimismo, el diseño e integración del algoritmo de Raíz Cuadrada mediante sumas y restas demostró el principio de abstracción hardware-software: limitaciones físicas en el set de instrucciones pueden

ser solventadas enteramente por la lógica programable. La identificación y corrección de bugs en el simulador ModelSim consolidan la base de este procesador, dejándolo completamente preparado para la integración de saltos y memoria de datos en sus fases posteriores.

VIII. Referencias

Harris, D. M., & Harris, S. L. (2012). *Digital Design and Computer Architecture* (2.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780123944245/digital-design-and-computer-architecture>

Patterson, D. A., & Hennessy, J. L. (2013). *Computer Organization and Design MIPS Edition: The Hardware/Software Interface* (5.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780124077263/computer-organization-and-design>

Sweetman, D. (2006). *See MIPS Run* (2.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780120884216/see-mips-run>

Universidad de Washington. (s.f.). *MIPS Instruction Reference*. Paul G. Allen School of Computer Science & Engineering. https://courses.cs.washington.edu/courses/cse378/01au/mips_instruction_reference.html